



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI INFORMATICA - SCIENZA E INGEGNERIA

CORSO DI LAUREA TRIENNALE IN
INGEGNERIA INFORMATICA

PROGETTAZIONE E SVILUPPO DI UN AMBIENTE
INTEGRATO PER LA GENERAZIONE E LA
RISOLUZIONE DI PROBLEMI DI RICERCA NELLO
SPAZIO DEGLI STATI

Tesi in Fondamenti di Informatica T-1

Relatore
Prof. Federico Chesani

Presentata da
Gabriele Doti

Co-relatore
Ing. Luca Giuliani

Sessione di Luglio 2025

Anno Accademico 2024/2025

Abstract

Questo progetto, articolato in 3 task, si propone di facilitare la definizione, generazione e risoluzione di problemi di ricerca su grafi orientati tramite un sistema integrato. Il primo task prevede lo sviluppo di un'interfaccia grafica che sostituisce la scrittura manuale di file YAML, facilitando l'inserimento di nodi, archi, costi ed euristiche. Il secondo task riguarda la generazione automatica di tracce d'esame configurabili tramite interfaccia, con specifica di parametri quali numero, difficoltà e consistenza euristica. Il terzo task implementa un sistema di riconoscimento da immagini di grafi orientati, capace di interpretare il problema rappresentato e fornire la soluzione tramite algoritmi Depth-First e A*. L'intero sistema è pensato per rendere più accessibile e flessibile la creazione e la risoluzione di esercizi basati su grafi, semplificandone la gestione e riducendone le complessità operative legate alla loro definizione e risoluzione.

Indice

1	Fondamenti teorici	3
1.1	Problemi di Ricerca negli Spazi degli Stati	3
1.2	Problemi di Ricerca: componenti e formalizzazione	3
1.3	Problemi a stati singoli e a stati multipli	4
1.4	Strategie di ricerca	4
1.4.1	Strategia di ricerca: informate e non informate	5
1.4.2	Valutazione delle strategie: completezza, ottimalità, tempo e spazio	5
1.5	Strategie di ricerca non informate	5
1.5.1	Strategie di ricerca informate	7
1.6	Funzioni euristiche: ammissibilità e consistenza	8
2	Analisi del Problema	11
2.1	Obiettivi della tesi	11
2.2	Specifiche dei tre task principali	11
2.3	Utente Target	12
2.4	Requisiti funzionali	12
2.5	Requisiti non funzionali	14
2.6	Vincoli	14
3	Progetto e Implementazione	17
3.1	Architettura generale dell'applicazione	17
3.2	Interfaccia grafica: scelte tecniche e componenti	18
3.3	Generazione dei problemi	19
3.4	Analisi di immagini	21
3.5	Modifiche al codice della libreria esterna	22
3.6	Costruzione e salvataggio dei file YAML	23
3.7	Invocazione del programma esterno e raccolta risultati	23
4	Esperimenti e Risultati	27
4.1	Esempi di problemi generati	27
4.2	Esempi di immagini analizzate	30
4.3	Analisi delle prestazioni	34

Introduzione

Contesto e Motivazione

La risoluzione dei problemi di ricerca è un argomento cruciale nell'intelligenza artificiale e nella teoria dei grafi. Questi riguardano, nel caso analizzato, il calcolo del cammino più breve fra 2 nodi di un grafo, verosimilmente fra una sorgente, *source*, punto di partenza, e una destinazione, *goal*, obiettivo e punto di arrivo. Le strategie di ricerca esistenti sono molteplici e vengono utilizzate in vari campi, che vanno dalla robotica alla logistica, passando per la gestione del traffico e i sistemi di navigazione. La natura dei problemi affrontata è però astratta, in quanto questi non hanno un'applicazione pratica diretta, ma sono stati pensati per risolvere situazioni ipotetiche o teoriche. Ciò è dovuto alla natura intrinseca del problema, in quanto l'obiettivo è ottenere esercizi di risoluzione di problemi nello spazio degli stati da presentare in sede d'esame, di cui si richiede la risoluzione. L'attenzione si sposta quindi sugli aspetti più tecnici e teorici del problema, come la gestione di casi particolari, come ad esempio la selezione del nodo successivo in presenza di più nodi con lo stesso valore di priorità.

Obiettivi del Progetto

Il punto di partenza è una libreria esterna, che, utilizzando un file YAML fornito come input, creato manualmente, contenente i dati del problema, restituisce un file DOCX con le soluzioni, applicando vari algoritmi di ricerca su grafi per risolvere problemi di ricerca spaziale. Il contributo apportato in questo elaborato mira innanzitutto a facilitare l'utilizzo del programma tramite l'implementazione di interfacce grafiche, assenti nella versione originale, che arricchiscono il progetto esistente migliorando l'esperienza utente. Un secondo contributo rilevante riguarda la riorganizzazione e l'estensione del codice, con modifiche agli algoritmi per gestire casi particolari e garantire una maggiore riutilizzabilità. Infine, è stata introdotta la generazione casuale di problemi, una funzionalità assente nel programma di base, che si limitava a fornire soluzioni senza occuparsi della creazione degli esercizi. La generazione randomica dei problemi è introdotta da un'interfaccia grafica che permette di settare il grado di difficoltà dei problemi generati, e fornisce in output sia le soluzioni, applicando vari algoritmi di ricerca e riportando i risultati in un file DOCX, sia il file YAML associato, nell'ottica di una modifica manuale dei dati del problema rispetto all'esercizio proposto, per generare un esercizio più adeguato alle proprie richieste, partendo da un valido punto di partenza. Infine, il progetto

si conclude con uno sviluppo più ambizioso: l'integrazione di un sistema in grado di analizzare un'immagine contenente un grafo e di ricostruire automaticamente il problema corrispondente, per poi risolverlo. Questa funzionalità, ancora in fase di sviluppo, rappresenta un possibile ampliamento futuro del progetto, volto a rendere lo strumento ancora più versatile e accessibile.

Struttura della tesi

La tesi è strutturata in cinque capitoli. Nel Capitolo 1 viene fornito il contesto e le basi teoriche necessarie per comprendere il lavoro svolto. Il Capitolo 2 analizza il problema, esaminandolo sia sotto l'aspetto funzionale sia sotto quello non funzionale. Nel Capitolo 3 viene descritto il progetto, illustrando le scelte progettuali e l'implementazione effettuata. Il Capitolo 4 presenta gli esperimenti condotti e i risultati ottenuti, mentre il Capitolo 5 conclude con una sintesi delle principali considerazioni e suggerisce possibili sviluppi futuri.

Metodologia

Le richieste emerse durante l'analisi del problema hanno portato alla suddivisione del progetto in tre componenti principali, ciascuna delle quali affronta un aspetto specifico, come descritto nel paragrafo 2.2. Per ognuna di queste componenti è stato seguito un approccio strutturato, articolato nelle seguenti fasi: raccolta dei requisiti, progettazione e implementazione, testing e valutazione. Ogni fase è stata condotta tenendo conto della coerenza complessiva del sistema e dell'integrazione tra le varie componenti.

Capitolo 1

Fondamenti teorici

1.1 Problemi di Ricerca negli Spazi degli Stati

La componente fondamentale della maggior parte dei problemi di Intelligenza Artificiale simbolica è la ricerca. In particolare, tale ricerca viene eseguita entro i confini di uno **spazio degli stati**, ovvero una rappresentazione astratta e formale creata per la modellazione dei problemi. Tali problemi assumono il nome di *Problemi di Ricerca*.

1.2 Problemi di Ricerca: componenti e formalizzazione

L'insieme degli stati possibili compone lo **Spazio degli Stati**, nei quali, attraverso una serie di trasformazioni da uno stato iniziale a uno stato finale, si esegue la ricerca.

Si definiscono quindi le seguenti componenti:

- **Stato**: configurazione del sistema in un certo momento. Può essere rappresentato da un nodo all'interno di un grafo.
- **Stato iniziale**: stato di partenza del problema.
- **Stato finale**: insieme di stati che soddisfano la soluzione.
- **Spazio degli stati**: insieme di tutti gli stati raggiungibili dallo stato iniziale tramite una sequenza finita di applicazioni degli operatori. Può essere rappresentato come un grafo orientato o un albero.
- **Azione o Funzione Successore**: insieme delle possibili transizioni verso nuovi stati, rappresentate come coppie $\langle a, x' \rangle$, dove:
 - a è un'azione applicabile in x ,
 - x' è lo stato risultante dopo l'applicazione di a .
- **Funzione di costo**: funzione $c(x, a, x')$ che associa un valore non negativo all'azione a eseguita nello stato x che conduce allo stato x' .
- **Soluzione**: sequenza di azioni che portano dallo stato iniziale al goal.

1.3 Problemi a stati singoli e a stati multipli

Nell'ambito della modellazione dei problemi è possibile distinguere i problemi che si differenziano in base al grado di conoscenza che ha l'agente dello stato in cui si trova.

Per chiarire meglio questo concetto, partiamo dalla definizione di agente.

Secondo [4], un **agente** è un'entità, fisica o virtuale, che percepisce l'ambiente in cui opera attraverso sensori e agisce su di esso tramite attuatori, con l'obiettivo di raggiungere uno scopo. A ogni istante, l'agente decide quale azione compiere in base alle informazioni raccolte fino a quel momento.

Esistono quindi due categorie di problemi:

- **Problemi a stati singoli**, in cui lo stato è sempre accessibile e l'agente conosce a priori il prodotto delle sue azioni, e può dunque calcolare in ogni momento in quale stato si troverà dopo una qualunque sequenza di azioni.
- **Problemi a stati multipli**, in cui lo stato non è completamente accessibile, dunque l'agente può non conoscere a priori il risultato delle sue azioni, per cui viene richiesta, durante l'esecuzione, una capacità di rilevamento, al fine di ragionare sui possibili stati raggiungibili ed evitare il raggiungimento di stati imprevisi o sconosciuti.

1.4 Strategie di ricerca

La ricerca è il perno dei Problemi di Ricerca: consiste nel processo che ha come fine l'individuazione della migliore sequenza di azioni che conducono a stati di esito conosciuto. Solo una volta trovata la migliore sequenza di azioni si passa alla fase dell'**Esecuzione**, ovvero la fase in cui le azioni suggerite vengono applicate.

Per una comprensione più chiara dei concetti che seguiranno, è necessario introdurre alcune definizioni fondamentali:

- **Espansione**: processo che consiste nella ricerca di nuovi stati, in cui ogni stato rappresenta una possibile configurazione successiva all'applicazione di un'azione.
- **Strategia di ricerca**: logica che guida l'agente nella corretta applicazione dell'espansione dei nodi, responsabile dell'ordine della scelta dei nodi.
- **Albero di ricerca**: rappresentazione ordinata dell'espansione dei nodi, dove a partire da un nodo di partenza, radice dell'albero, vengono mostrati tutti i nodi adiacenti, ovvero quelli che è possibile espandere dallo stato corrente. L'espansione è riservata ad un nodo per volta e riguarda le foglie dell'albero corrente. Questo processo continua fino al raggiungimento del *goal*, ossia fino a quando non viene trovato uno stato finale che soddisfa i criteri di soluzione.

L'albero di ricerca è quindi una precisa struttura dati formata da nodi, anch'essi definiti da una struttura dati che contiene:

- Lo stato a cui corrisponde il nodo nello spazio degli stati
- Il nodo genitore
- L'operatore che è stato applicato per ottenere il nodo
- La profondità del nodo
- Il costo del cammino, dallo stato iniziale fino al nodo corrente

1.4.1 Strategia di ricerca: informate e non informate

Nell'espansione di un nodo, l'agente si serve di una strategia, che rappresenta la scelta di quale nodo espandere. Le strategie di ricerca si dividono in due categorie:

- **Strategie di ricerca non informate** (o *blind*): non utilizzano alcuna conoscenza euristica nel determinare il nodo da espandere.
- **Strategie di ricerca informate** (o *euristiche*): utilizzano conoscenza euristica sul problema per la selezione degli operatori applicabili.

1.4.2 Valutazione delle strategie: completezza, ottimalità, tempo e spazio

Le strategie applicabili in un Problema degli Stati sono molteplici, ed è possibile confrontarle fra di loro in base a quattro criteri di valutazione:

- **Completezza**: capacità della strategia di garantire il raggiungimento di una soluzione, quando ne esiste almeno una.
- **Complessità temporale**: complessità della strategia in relazione al tempo che occorre per trovare una soluzione.
- **Complessità spaziale**: complessità della strategia in relazione alla memoria che occorre impiegare per trovare una soluzione.
- **Ottimalità**: capacità della strategia di raggiungere la migliore soluzione in presenza di molteplici soluzioni.

1.5 Strategie di ricerca non informate

Le strategie di ricerca non informate determinano il successivo nodo da espandere non seguendo alcuna euristica, ma esclusivamente in base alla struttura dell'albero di ricerca. Sono quindi strettamente legate alla struttura dati che rappresenta il problema.

Si possono elencare due principali strategie di ricerca non informate:

- **Breadth-first**;

- **Depth-first.**

Per descrivere tali strategie di ricerca, si fornisce una definizione di *profondità*: la profondità di un certo nodo in un dato albero è rappresentata dal numero di archi che separano il nodo dalla radice. Si assegna dunque al nodo radice un valore di profondità pari a zero.

Breadth-first La strategia di ricerca *Breadth-first* consiste nell'espandere sempre prima i nodi meno profondi dell'albero. Questa strategia procede quindi per livelli, partendo dalla radice (profondità 0) e passando ai nodi via via più profondi.

Indichiamo con b il massimo fattore di diramazione dell'albero e con d la profondità della soluzione a costo minimo. La strategia può essere valutata secondo i seguenti criteri:

- **Completezza:** garantita se b è finito.
- **Tempo:** $O(b^d)$, dato che i nodi vengono espansi livello per livello: $1+b+b^2+\dots+b^d$.
- **Spazio:** anch'esso $O(b^d)$, poiché tutti i nodi generati devono essere mantenuti in memoria.
- **Ottimalità:** sì, se il costo è unitario per ogni passo; no, in generale.

La Breadth-first è molto dispendiosa in termini di memoria e tempo se la soluzione si trova a grande profondità. Inoltre, la completezza non è garantita se b è infinito, in quanto si rischia una ricerca infinita su un singolo livello. L'ottimalità non è garantita se i costi non sono unitari.

Depth-first La strategia *Depth-first* espande prima i nodi più profondi. A parità di profondità, vengono scelti i nodi più a sinistra.

Indichiamo con b il massimo fattore di diramazione e con m la massima profondità dello spazio degli stati. La strategia presenta le seguenti caratteristiche:

- **Completezza:** no, non è completa in generale. In presenza di un ramo infinito, l'algoritmo potrebbe esplorarlo indefinitamente.
- **Tempo:** nel caso peggiore, può esplorare tutti i nodi fino alla profondità massima m : $O(b^m)$.
- **Spazio:** solo il cammino corrente e i fratelli non ancora esplorati vengono mantenuti, per cui lo spazio richiesto è $O(bm)$.
- **Ottimalità:** no, può restituire soluzioni non ottimali.

La Depth-first è vantaggiosa dal punto di vista dello spazio, ma non garantisce completezza né ottimalità. È adatta a problemi con spazi di ricerca molto ampi ma con soluzioni a profondità contenuta e distribuite regolarmente.

Depth-first a profondità limitata La ricerca a profondità limitata è una variante della Depth-first in cui viene fissata una profondità massima l oltre la quale i nodi non vengono esplorati.

Se il goal non viene trovato entro la profondità l , l'agente considera nodi alternativi alla stessa profondità oppure ritorna a nodi di profondità inferiore. Questa strategia è in grado di gestire loop o rami infiniti, evitando l'esplorazione infinita di cammini senza soluzione.

1.5.1 Strategie di ricerca informate

Le strategie di ricerca informate vengono utilizzate in problemi di grandi dimensioni, nei quali la complessità spaziale e temporale delle strategie non informate risulta eccessiva. In questo contesto, la scelta del nodo da espandere viene effettuata sulla base di una funzione euristica, sfruttando quindi la conoscenza specifica del dominio del problema.

L'elemento fondamentale delle strategie informate è la *funzione di valutazione*, la cui scelta incide fortemente sull'efficacia della ricerca. Una funzione ben progettata può fornire informazioni più precise e puntuali, facilitando il raggiungimento di soluzioni più efficaci.

Ricerca Best-First La strategia *Best-First* consiste nello scegliere come prossimo nodo da espandere quello che sembra più promettente secondo una funzione di valutazione. Questa funzione assegna un valore numerico a ciascun nodo, e l'espansione prosegue in direzione del nodo che appare più desiderabile, nell'ottica di raggiungere la soluzione.

A seconda della funzione di valutazione adottata, si distinguono due casi particolari:

- Ricerca *Greedy Best-First*
- Ricerca A^*

Ricerca Greedy Best-First Nella strategia *Greedy Best-First*, la funzione di valutazione stima la distanza del nodo attuale dal goal. Non viene però considerata la distanza percorsa per raggiungere quel nodo, ma solamente la distanza stimata residua.

Pertanto, questa strategia non garantisce l'ottimalità della soluzione: può portare l'agente verso un nodo che appare vicino alla meta, ma che si rivela costoso da raggiungere effettivamente.

La funzione di valutazione utilizzata è:

$$f(n) = h(n)$$

dove:

- $f(n)$ è la funzione di valutazione;
- $h(n)$ è la funzione euristica, che stima la distanza del nodo n dal goal.

Ricerca A^* La strategia A^* rappresenta un'evoluzione della Best-First. Essa tiene conto sia della distanza già percorsa per raggiungere il nodo corrente, sia della stima della distanza restante fino al goal. In questo modo, A^* combina i punti di forza della ricerca greedy e della ricerca a costo uniforme.

La funzione di valutazione è definita come:

$$f(n) = g(n) + h(n)$$

dove:

- $g(n)$ è il costo (o profondità) del cammino percorso dall'agente fino al nodo n ;
- $h(n)$ è la stima del costo per raggiungere il goal da n .

Il punto di forza della ricerca A^* è la sua capacità di combinare l'efficienza della ricerca in profondità con l'ottimalità e la completezza garantite dalla ricerca a costo uniforme.

1.6 Funzioni euristiche: ammissibilità e consistenza

Nell'ambito dell'algoritmo A^* , è possibile mettere in relazione il raggiungimento di una soluzione con determinate proprietà dell'euristica, quali ammissibilità e consistenza.

Ammissibilità Secondo la definizione data da [4], un'euristica è **ammissibile** se ogni nodo non sovrastima mai il costo minimo necessario per raggiungere lo stato obiettivo, a partire da quel nodo. Quindi è necessario che, per ogni nodo, si abbia sempre:

$$h(n) \leq h^*(n)$$

dove:

- $h(n)$ è la funzione euristica,
- $h^*(n)$ è il costo reale del percorso più economico.

In particolare, l'algoritmo A^* è ottimo rispetto al costo se utilizza un'euristica ammissibile.

Consistenza Secondo la definizione data da [4], un'euristica è **consistente** se per ogni nodo n e ogni successore n' , generato in seguito all'espansione di n dovuta ad un'azione a , si ha:

$$h(n) \leq c(n, a, n') + h(n')$$

dove:

- $h(n)$ è la funzione euristica di n ,
- $c(n, a, n')$ è il costo per passare dal nodo n al nodo n' ,
- $h(n')$ è la funzione euristica di n' .

Considerazioni su ammissibilità e consistenza Mentre un'euristica ammissibile rende A^* ottimale dal punto di vista del costo, questo non è garantito nel caso contrario.

Come riportato da [4], esistono due casi in cui l'algoritmo A^* restituisce comunque una soluzione ottima, anche se l'euristica non è strettamente ammissibile:

1. Se esiste almeno un percorso ottimale (cioè a costo minimo) lungo il quale l'euristica $h(n)$ è ammissibile per tutti i nodi n su quel percorso, allora A^* troverà comunque quel percorso ottimale, indipendentemente dai valori dell'euristica negli stati che non appartengono a quel cammino.
2. Se la soluzione ottimale ha costo C^* e la seconda migliore ha costo C_2 , e se l'euristica $h(n)$ sovrastima alcuni costi, ma mai di più di $C_2 - C^*$, allora A^* è comunque garantito restituire una soluzione ottimale.

Inoltre, ammissibilità e consistenza sono strettamente legate fra di loro: se un'euristica è consistente, essa è anche ammissibile. Dunque non è possibile avere un'euristica consistente ma non ammissibile. Non è invece garantito il contrario.

Capitolo 2

Analisi del Problema

2.1 Obiettivi della tesi

Il punto di partenza di questo progetto è una libreria esterna in grado di risolvere problemi di ricerca nello spazio degli stati utilizzando l'algoritmo A^* . Esso richiede in input un documento in formato YAML, che deve essere scritto manualmente rispettando una struttura ben definita. A partire da questo file, il programma è in grado di elaborare la soluzione dell'esercizio, ovvero applicare prima l'algoritmo Depth-First, e poi l'algoritmo A^* , se i dati forniti in input rappresentano un problema ammissibile. Tale soluzione viene poi adeguatamente formattata in un file in formato DOCX, che rappresenta l'output del programma.

2.2 Specifica dei tre task principali

L'obiettivo della tesi si articola in tre compiti principali, ciascuno finalizzato a migliorare l'esperienza utente, arricchendola di funzionalità non presenti nella versione originaria del sistema. Da questo momento in poi, per comodità espositiva, tali richieste verranno indicate rispettivamente come **Task 1**, **Task 2** e **Task 3**.

- **Task 1** – Il primo task prevede la realizzazione di un'interfaccia utente che migliori l'esperienza complessiva, sostituendo la scrittura manuale del file YAML con un sistema guidato per l'inserimento dei dati relativi all'esercizio. In questo modo, l'utente può specificare nodi, archi, costi, valori euristici e obiettivi direttamente dall'interfaccia grafica, riducendo il rischio di errori di sintassi o di struttura e semplificando l'interazione con il programma sottostante, che inizialmente richiedeva l'uso del terminale.
- **Task 2** – Il secondo task riguarda la creazione di un sistema dedicato alla generazione automatica di tracce d'esame su problemi di ricerca. Anche in questo caso, l'interazione con il sistema è facilitata da un'apposita interfaccia grafica, che consente all'utente di modellare le caratteristiche delle tracce da generare, specificando il numero desiderato, il livello di difficoltà e la consistenza dell'euristica.

- **Task 3** – Il terzo task consiste nell’elaborazione automatica di un’immagine raffigurante un grafo orientato che rappresenta un esercizio da risolvere mediante gli algoritmi Depth-First e A*. Il sistema è in grado di leggere e analizzare il contenuto dell’immagine, e successivamente fornire la soluzione in formato DOCX.

2.3 Utente Target

L’utente target principale del sistema è il docente universitario, al quale viene fornito un programma per la correzione e la creazione di esercizi sulla risoluzione di problemi di ricerca, con particolare riferimento all’algoritmo A*. Questo strumento è pensato per essere impiegato durante prove d’esame o esercitazioni pratiche, sia per generare problemi nuovi in modo automatico, sia per verificarne la correttezza e difficoltà. Inoltre vi è un coinvolgimento secondario anche da parte degli studenti universitari: pur non essendo destinato al loro utilizzo diretto, lo strumento ha un impatto significativo sull’esperienza formativa, in quanto consente ai docenti di proporre esercizi strutturati, corretti e vari, migliorando la qualità complessiva delle prove d’esame.

2.4 Requisiti funzionali

Task 1: Creazione manuale di un problema tramite interfaccia grafica

- **RF1.1 - Inserimento dei dati tramite interfaccia:** Il sistema deve fornire un’interfaccia grafica che consenta all’utente di inserire interattivamente le informazioni necessarie alla definizione di un problema (nodi, archi, costi, euristiche, stato iniziale e finale, titolo, data e parametri grafici).
- **RF1.2 - Validazione dei dati inseriti:** L’interfaccia deve includere controlli che impediscano l’inserimento di dati incoerenti o incompleti, evitando errori di sintassi o struttura.
- **RF1.3 - Generazione automatica del file YAML:** I dati inseriti devono essere convertiti in un file YAML valido, conforme al formato richiesto dal programma di risoluzione.
- **RF1.4 - Invocazione automatica del risolutore:** Una volta generato il file YAML, l’interfaccia deve eseguire automaticamente il programma esterno passando il file come input.

Task 2: Generazione automatica di problemi tramite parametri

- **RF2.1 - Definizione dei parametri di generazione:** L’interfaccia deve permettere all’utente di selezionare:
 - il numero di problemi da generare;
 - il livello di difficoltà desiderato (facile, medio, difficile);

- se l'euristica deve essere consistente oppure no.
- **RF2.2 - Generazione automatica dei problemi:** Il sistema deve essere in grado di generare automaticamente problemi validi sulla base dei parametri scelti, scartando quelli che non rispettano i vincoli.
- **RF2.3 - Visualizzazione della struttura dei risultati:** Una volta completata la generazione, l'interfaccia deve mostrare la struttura delle cartelle create, ognuna contenente un file `.yaml` e un file `.docx` per ciascun problema.
- **RF2.4 - Esportazione dei problemi:** Tutti i problemi validi devono essere salvati all'interno di una directory denominata `exams`, con sottocartelle `result`, `result(1)`, `result(2)` ecc., in sottocartelle `result_1`, `result_2`, ecc., contenenti i relativi file `.yaml` e `.docx`.

Task 3: Analisi automatica di un'immagine PNG contenente un problema

- **RF3.1 - Input del percorso dell'immagine via terminale:** Il sistema deve accettare da linea di comando il path a un file in formato PNG, che rappresenta graficamente un problema di ricerca.
- **RF3.2 - Riconoscimento automatico dei componenti del grafo:** Il sistema deve analizzare l'immagine e identificare automaticamente:
 - i nodi del grafo;
 - gli archi, compresa l'eventuale direzione;
 - i costi associati agli archi;
 - le euristiche associate ai nodi.
- **RF3.3 - Interpretazione semantica del problema:** Le informazioni estratte devono essere strutturate internamente in modo coerente, per consentire la successiva fase di risoluzione.
- **RF3.4 - Risoluzione automatica del problema:** Il sistema deve risolvere il problema utilizzando i seguenti algoritmi:
 - Depth-First Search (DFS);
 - A* Search.
- **RF3.5 - Generazione automatica della soluzione in formato DOCX:** Il sistema deve esportare le soluzioni trovate in un file DOCX.

2.5 Requisiti non funzionali

Usabilità

- Le interfacce grafiche devono essere chiare e facilmente comprensibili, con elementi visivi intuitivi per evitare errori nell’inserimento dei dati.
- Il sistema deve essere fruibile anche da utenti non esperti, con un’interazione semplificata rispetto alla scrittura manuale dei file YAML.

Affidabilità

- Il programma deve garantire l’elaborazione accurata dei dati in ingresso (sia tramite interfaccia che tramite immagine).
- I risultati (file YAML, esercizi generati, soluzioni in DOCX) devono essere corretti e consistenti.

Prestazioni

- L’interfaccia grafica deve rispondere rapidamente agli input dell’utente, con tempi di elaborazione ridotti per la generazione degli esercizi e l’analisi dei grafi da immagine.
- La creazione dei file DOCX deve avvenire in modo efficiente, senza bloccare l’interfaccia durante la generazione dei risultati.

Compatibilità

- Il sistema deve essere compatibile con Python e JupyterLab, utilizzando librerie comuni per la gestione dei file YAML, l’analisi delle immagini e la creazione dei documenti DOCX.
- Il programma deve essere compatibile con almeno il formato di immagine PNG.

2.6 Vincoli

Tecnologie

- Il sistema deve essere sviluppato in Python, utilizzando librerie adeguate per la gestione delle immagini e la generazione di file YAML.
- L’interfaccia grafica dovrà essere snella e non richiedere elevate risorse di sistema, utilizzando un framework leggero per garantire un’esperienza utente fluida.

Punto di partenza

- La libreria esterna è stata sviluppata per l'interazione tramite terminale e accetta file YAML come input. Il nuovo sistema deve integrarsi essa, mantenendo la compatibilità con l'input YAML esistente.

Interazione tramite YAML

- Nonostante l'interfaccia grafica, l'output del sistema deve essere in formato YAML, che poi verrà passato al programma di risoluzione degli algoritmi.
- In aggiunta, il terzo task comporta l'analisi di immagini, ma alla fine il sistema dovrà essere in grado di generare prima un file YAML e poi un file DOCX con la soluzione.

Capitolo 3

Progetto e Implementazione

3.1 Architettura generale dell'applicazione

Il programma di base fornito dal docente è stato sviluppato in Python, pertanto si è scelto di mantenere lo stesso linguaggio per garantire coerenza strutturale e interoperabilità tra i diversi componenti del sistema.

Per quanto riguarda gli algoritmi di ricerca, la libreria esterna fornisce già l'implementazione di alcuni di essi, quali Depth-first e A*, sufficienti per gli scopi del programma finale. Dunque, l'obiettivo non è implementare altri algoritmi di ricerca, quanto più modificare quelli esistenti affinché si adattino ai nuovi requisiti progettuali.

Inoltre, oltre allo sviluppo di soluzioni per le nuove richieste, viene fornita anche una nuova modalità di input/output: un'interfaccia grafica, che permette di facilitare l'interazione fra l'utente e il programma di base.

Ciò comporta una riconfigurazione strutturale del programma, poiché la logica applicativa deve interfacciarsi con i componenti della UI. Infine, nel programma finale, concepito per essere utilizzato tramite interfaccia grafica, perde di centralità lo script `main.py`, che costituiva il nucleo della libreria esterna. Tale script, se eseguito autonomamente, permetteva di ottenere la risoluzione del problema descritto da un file YAML in input, con output in formato DOCX. L'introduzione della UI comporta quindi una ridefinizione del flusso di esecuzione, spostando il focus dall'esecuzione diretta dello script a un'interazione mediata dall'interfaccia grafica.

Ciò non preclude però all'utente di interagire con il programma mediante terminale, eseguendo manualmente `main.py`, per cui, in generale, viene conservata la struttura del programma originale.

La presenza di interfacce grafiche comporta la creazione di un nuovo package, `ui`, il quale contiene gli script che ne descrivono il funzionamento. Gli script eseguiti in seguito all'interazione con l'utente, invece, vengono raggruppati in un altro nuovo package, `scripts`, compreso il file `main.py`. Infine, ai già presenti package di persistenza `sources` e `exports`, viene aggiunto il package `exams`, destinato a raccogliere l'output prodotto tramite l'interazione con l'interfaccia grafica elaborata nel **Task 2** del progetto.

Viene quindi adottata questa divisione in Model, Controller e Persistenza, che permette una netta separazione dei compiti, rappresentata graficamente nella figura sottostante.

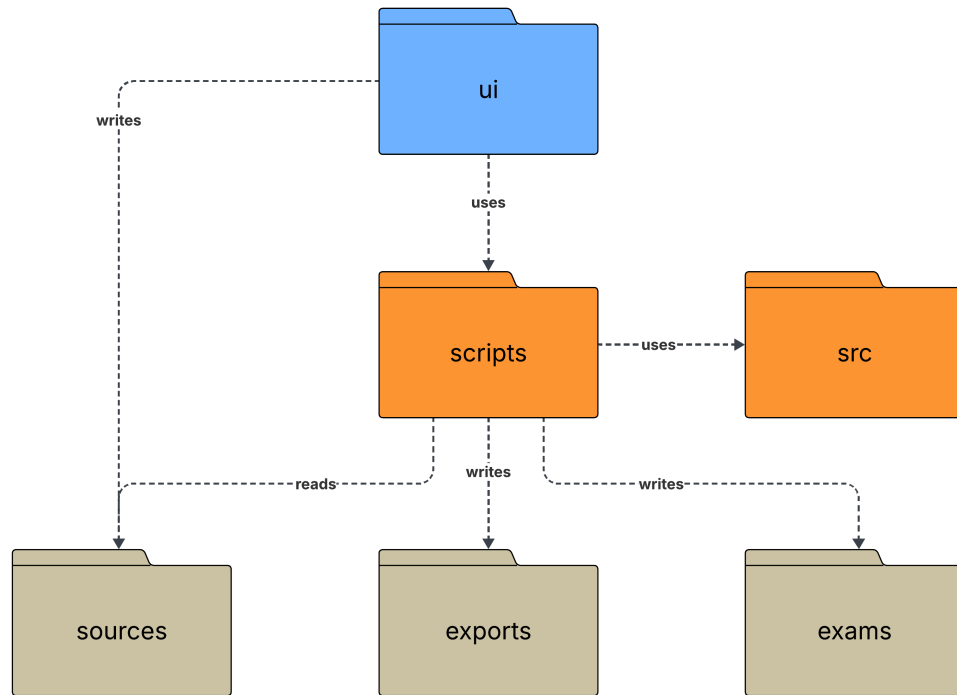


Figura 3.1: Divisione in packages dell'applicazione

3.2 Interfaccia grafica: scelte tecniche e componenti

Per lo sviluppo e la visualizzazione dell'interfaccia grafica si è scelto di utilizzare JupyterLab, un ambiente di sviluppo interattivo (IDE) open source basato su web [1]. JupyterLab consente la gestione strutturata dei notebook, ovvero documenti interattivi che combinano codice eseguibile, testo descrittivo e visualizzazioni in un unico file, con estensione `.ipynb`.

Le ragioni principali che hanno spinto all'utilizzo di JupyterLab sono le seguenti:

1. **Velocità di sviluppo:** grazie all'interattività del notebook, ogni modifica al codice si riflette immediatamente nell'interfaccia, velocizzando il ciclo di test.
2. **Portabilità:** i file con estensione `.ipynb` sono leggibili e modificabili in qualsiasi ambiente JupyterLab.

3. **Modularità:** i widget possono essere combinati e riutilizzati con facilità per creare interfacce più complesse.
4. **Visualizzazione integrata:** la possibilità di mostrare grafici, messaggi e file direttamente nel notebook facilita il debug e la verifica dei dati.

Per la creazione delle interfacce interattive si è optato per l'utilizzo della libreria **ipywidgets**, per via della perfetta integrazione nativa con l'ambiente Jupyter. Ciò ha permesso di realizzare elementi interattivi direttamente nelle celle dei notebook, senza la necessità di configurare ulteriori framework frontend.

Inoltre, la scelta è ricaduta su **ipywidgets** per via della sua facilità di utilizzo: la libreria permette di creare widget di vario tipo con poche righe di codice Python. Un ulteriore vantaggio è dato dalla possibilità di collegare i widget direttamente alle variabili e funzioni Python, con aggiornamenti dinamici e bidirezionali dello stato dell'interfaccia e dei dati sottostanti, fondamentale per processare i dati forniti in input dall'utente.

La disposizione degli elementi di input è semplice e intuitiva:

- **Task 1** – Nella prima interfaccia grafica, l'aggiunta manuale di nodi innesca la creazione delle celle di input per l'euristica e degli archi. È possibile definire il nodo sorgente e quello di destinazione, e impostare una serie di parametri che definiscono l'impaginazione del grafico nel file di output in formato `.docx`. Inoltre, è possibile scegliere, per l'algoritmo A^* , il metodo di espansione in caso di parità di $f(n)$. Nel caso in cui si selezioni l'opzione 'ordine nei nodi', viene aggiunto un menu a tendina per ogni nodo, contenente una lista numerica da 1 al numero totale dei nodi, utile a definire l'ordine di espansione.
- **Task 2** – Nella seconda interfaccia grafica è possibile impostare il numero e la difficoltà dei problemi da generare, nonché specificare se essi debbano essere consistenti o meno. È inoltre prevista la possibilità di generare problemi con difficoltà e/o consistenza miste.

I messaggi di errore vengono generati quando non tutti i campi di input necessari per l'esecuzione del programma sono stati riempiti, e sono visualizzabili nei log del corrispondente notebook.

3.3 Generazione dei problemi

Per la generazione randomica dei problemi si è deciso di adottare la strategia *generate-and-test*, secondo la quale si generano più candidati, si verifica se ciascun candidato rispetta uno o più vincoli o criteri, e infine si scartano i candidati che non rispettano i requisiti.

Per la generazione automatica di grafi casuali è stata utilizzata la libreria **networkx**. Questa libreria ha permesso di creare strutture grafiche orientate, alla quale è stata abbinata la logica necessaria per soddisfare i vincoli richiesti. In particolare, in seguito

alla generazione di un grafo orientato, si è verificato che tale grafo rispettasse una serie di requisiti che è possibile classificare in due categorie:

- **Requisiti di struttura:**

- Il nodo G non deve risultare connesso esclusivamente al nodo A.
- Il nodo G può solo ricevere archi in ingresso
- La media del numero di archi per nodo è probabilisticamente inversamente proporzionale a n
- Ogni nodo (eccetto il nodo destinazione) deve avere almeno un arco in uscita
- Ogni nodo (eccetto il nodo sorgente) deve avere almeno un arco in ingresso
- Se un nodo ha un solo arco in ingresso e uno in uscita, questi non possono coinvolgere lo stesso nodo
- Il numero di archi deve essere maggiore di del numero dei nodi
- Ogni nodo deve essere coinvolto in almeno una soluzione
- Devono esistere almeno tre soluzioni possibili

- **Requisiti di dominio:**

- Il percorso diretto da A a G deve avere un costo probabilisticamente alto
- Il secondo arco del cammino minimo deve avere un costo probabilisticamente medio-alto
- Gli archi del cammino più lungo devono avere probabilisticamente un costo basso
- Tutti i restanti archi devono avere un costo basso
- Il nodo G deve poter essere raggiunto da tutti gli altri nodi
Questo requisito viene verificato applicando l'algoritmo di Dijkstra al grafo; vengono quindi scartati i grafi che non sono in grado di fornire un costo finito del percorso minimo da ogni nodo verso la destinazione.

Infine, nel caso in cui siano attivate le opzioni di *consistenza* o *difficoltà*, viene verificato che il grafo orientato ottenuto rispetti queste specifiche richieste dall'utente. In particolare, la difficoltà viene calcolata in base al numero di espansioni effettuate per arrivare alla soluzione:

- **Facile:** 3–4 espansioni
- **Media:** 5–7 espansioni
- **Difficile:** 8–11 espansioni

3.4 Analisi di immagini

Il terzo task del progetto presenta un livello di complessità superiore rispetto ai precedenti. L'obiettivo consiste nel risolvere un problema rappresentato graficamente all'interno di un'immagine, rappresentante un grafo orientato, attraverso l'applicazione di algoritmi di ricerca nello spazio degli stati.

Considerati i vincoli temporali e la complessità intrinseca del task, si è scelto di definire dei vincoli operativi chiari, con l'obiettivo di garantire un primo risultato concreto e funzionante.

Le assunzioni alla base del funzionamento sono le seguenti:

- L'immagine fornita in input deve essere in formato PNG.
- La struttura del grafo contenuto nell'immagine segue un formato rigoroso:
 - **Nodi**: rappresentati da cerchi, con il nome del nodo scritto all'interno del cerchio.
 - **Euristiche**: indicate all'interno di rettangoli, posti nelle vicinanze del nodo corrispondente.
 - **Archi**: rappresentati da frecce, che possono essere monodirezionali o bidirezionali.
 - **Costi**: il costo di ogni arco è scritto lungo la freccia corrispondente, a circa un terzo della lunghezza della freccia a partire dalla punta. In caso di frecce bidirezionali, ciascun verso ha un proprio costo indicato con la stessa logica. In entrambi i casi, la presenza del costo "spezza" visivamente la freccia in due o tre segmenti.

Questa struttura grafica non è casuale: ricalca esattamente quella utilizzata nei file DOCX generati come soluzione nei task precedenti, garantendo così coerenza, riutilizzabilità e continuità del sistema.

Uso di OpenCV

Per la realizzazione di questo task è stata utilizzata OpenCV (Open Source Computer Vision), una libreria open source largamente adottata per attività di visione artificiale e analisi di immagini. Essa offre un ampio insieme di strumenti, tra cui algoritmi e modelli predefiniti per il riconoscimento di oggetti, la segmentazione di immagini e il rilevamento di forme geometriche [2].

L'impiego di OpenCV è stato fondamentale per l'identificazione dei pattern visivi presenti nell'immagine, permettendo di classificare correttamente nodi, archi, nomi ed euristiche da utilizzare nella ricostruzione del problema.

Fasi dell'implementazione

L'analisi dell'immagine avviene attraverso una serie di passaggi strutturati:

- **Rilevamento dei contorni:** utilizzando le funzionalità di OpenCV, si identificano i contorni presenti nell'immagine.
 - I cerchi vengono individuati per riconoscere i nodi. Una volta isolato ciascun cerchio, si applica la tecnologia OCR (Optical Character Recognition) per leggere automaticamente il nome al suo interno. Come riportato in [3], l'OCR è una tecnologia che consente di estrarre dati da immagini di testo, convertendoli in un formato leggibile da dispositivi automatici.
 - I rettangoli, riconoscibili geometricamente, vengono associati alle euristiche. Il collegamento tra rettangolo e nodo avviene per prossimità spaziale.
- **Analisi degli archi:** l'individuazione delle frecce è realizzata tramite la rilevazione delle linee nell'immagine. Le linee vengono poi raggruppate secondo inclinazione e vicinanza, ricostruendo così le frecce.
 - Il verso della freccia è determinato sfruttando il comportamento di OpenCV nella rilevazione delle linee vicine alla punta.
 - Per ciascuna freccia, viene calcolata la posizione attesa del costo (a un terzo della lunghezza dalla punta), e si estrae l'area interessata per applicare l'OCR. Il numero letto viene infine associato all'arco corretto.

Una volta completata l'analisi, i dati raccolti (nodi, archi, euristiche, costi) vengono strutturati in un file YAML, che viene fornito in input allo script `main.py`.

Come nei task precedenti, il programma elabora il file e produce la soluzione in formato DOCX, applicando sia l'algoritmo di ricerca Depth-First Search che A*.

3.5 Modifiche al codice della libreria esterna

Per garantire il riutilizzo del codice, è stato necessario apportare alcune modifiche al codice della libreria esterna, in particolare al file `search.py` presente nel package `src`. Questo modulo applica gli algoritmi di ricerca ai dati di input forniti e produce una soluzione in un apposito file DOCX.

Sono stati effettuati cambiamenti in due principali aree del codice:

1. **Introduzione delle due modalità di ordinamento**
2. **Supporto alla verifica dei requisiti nella strategia generate-and-test**

1. Ordinamento a parità di $f(n)$ Nel programma originale, in caso di nodi con valori identici di $f(n)$, il nodo scelto per l'espansione era semplicemente l'ultimo inserito tra i candidati. Questa logica è stata modificata per gestire due nuove modalità di ordinamento:

- **Ordine alfabetico:** è stata introdotta una variabile per tenere traccia del nome del nodo, che in precedenza non veniva considerato nel confronto. In caso di parità su $f(n)$, viene quindi selezionato il nodo con nome alfabeticamente inferiore. A parità anche di nome, viene scelto il nodo espanso per primo.
- **Ordine dei nodi definito dall'utente:** è stato introdotto un dizionario che associa a ogni nodo la sua posizione nell'ordine imposto dall'utente. In caso di parità su $f(n)$, l'espansione segue questo ordine. Se anche l'ordine è identico, viene selezionato il nodo espanso per primo.

2. Disattivazione della generazione del file DOCX Nel codice originale, la ricerca era sempre seguita dalla generazione del file di output `.docx`. Questa operazione, pur utile per la produzione del risultato finale, introduce un ritardo non trascurabile (di alcuni secondi), soprattutto nel contesto della strategia *generate-and-test*, dove è necessario verificare rapidamente un grande numero di problemi generati casualmente.

Per questo motivo, è stata introdotta un'opzione che permette di eseguire la ricerca senza produrre il file `.docx`, rendendo il processo molto più efficiente nella fase di verifica automatica dei requisiti.

3.6 Costruzione e salvataggio dei file YAML

Per ottemperare a uno dei requisiti non funzionali fondamentali del progetto, ovvero la **riusabilità del codice**, si è deciso di sfruttare appieno le funzionalità dello script `main.py`, già presente nel programma originale.

In particolare, per garantirne l'utilizzo, è stato necessario creare un file YAML personalizzato, nonché unico formato di input accettato dallo script `main.py` a ogni sua esecuzione.

Nel **Task 1**, viene creato un singolo file YAML contenente i dati raccolti in input e successivamente esportato nella directory `exports`.

Nel **Task 2**, invece, vengono creati tanti file YAML quanti sono i tentativi effettuati per ottenere il numero corretto di problemi generati con le caratteristiche specificate dall'utente. In questo caso, i file YAML che descrivono problemi conformi ai requisiti vengono forniti ordinatamente in output all'utente, assieme al file DOCX che mostra la relativa soluzione.

3.7 Invocazione del programma esterno e raccolta risultati

Il sistema prevede due modalità operative, corrispondenti alle due interfacce grafiche. In entrambi i casi si invoca unoo più script esterni tramite la libreria `subprocess` di Python,

al fine di generare o risolvere esercizi basati su problemi di ricerca in uno spazio degli stati.

Task 1

Nel primo caso, in seguito all'interazione con l'utente, lo script si limita a invocare direttamente `main.py`, il quale genera un file `.docx` contenente la descrizione del problema e la sua soluzione.

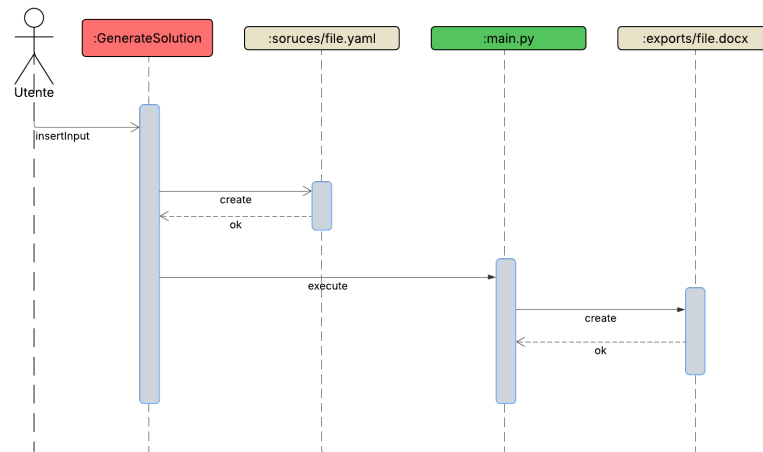


Figura 3.2: Diagramma delle interazioni per Task 1.

Task 2

Nel secondo caso, l'obiettivo è generare automaticamente un numero specifico di esercizi, indicato dall'utente. A tal fine, in seguito all'interazione dell'utente, si invoca lo script `generate_exams.py`, che si occupa di:

- invocare ripetutamente `main.py`;
- verificare se l'esercizio generato soddisfa i criteri richiesti;
- salvare ogni esercizio valido in una sottocartella distinta.

Ogni sottocartella contiene:

- un file `.yaml` con i dati del grafo;
- un file `.docx` con la soluzione del problema.

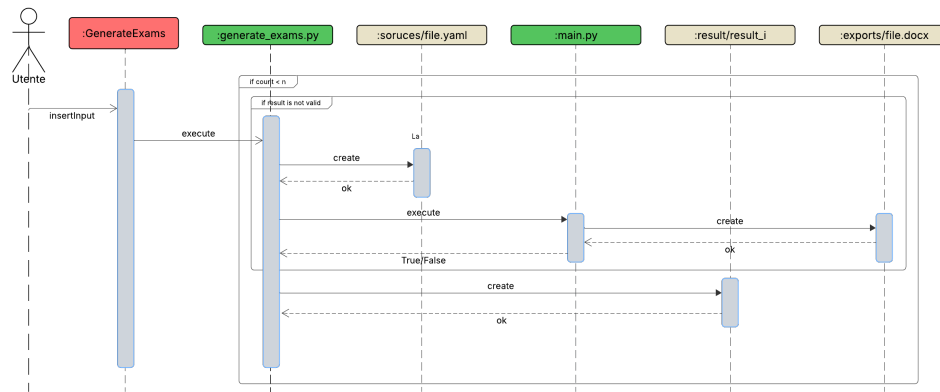


Figura 3.3: Diagramma delle interazioni per Task 2.

Task 3

Nel terzo caso, l'obiettivo è analizzare un'immagine illustrante un problema, captarne le componenti e fornirne una soluzione. A questo scopo, in seguito all'interazione dell'utente il quale fornisce in input il path di un file PNG, si invoca lo script `generate_exams.py`, che si occupa di generare la soluzione in formato DOCX.

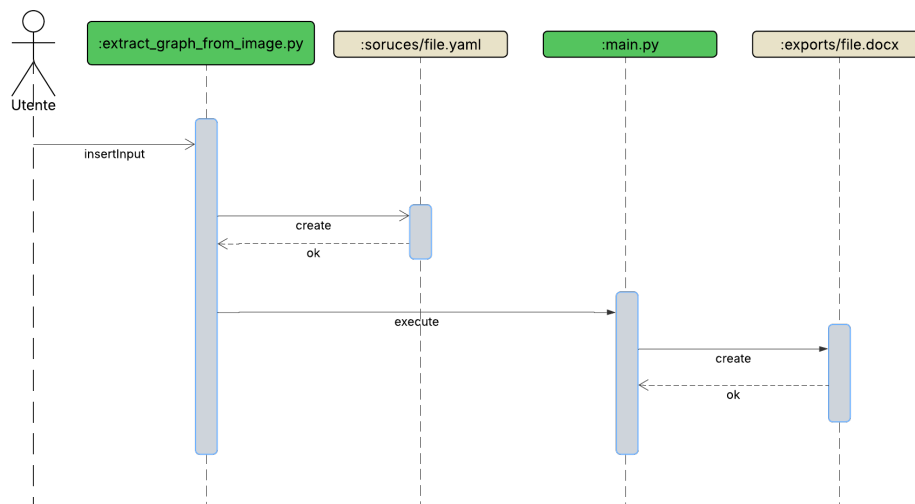


Figura 3.4: Diagramma delle interazioni per Task 3.

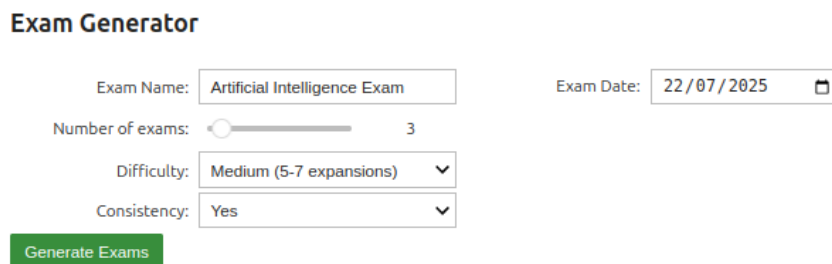
Capitolo 4

Esperimenti e Risultati

In questo capitolo l'attenzione viene spostata principalmente sul secondo e sul terzo task, in quanto sono quelli in cui il sistema sviluppato viene messo realmente alla prova in contesti automatici e semi-automatici, mostrando la sua capacità di generare, validare e risolvere problemi in modo efficiente e ripetibile.

4.1 Esempi di problemi generati

I problemi possono essere generati in base alla difficoltà e alla consistenza. Di seguito viene mostrato un esempio in cui si richiede la generazione di tre problemi consistenti di difficoltà media.



The image shows a web interface titled "Exam Generator". It contains several input fields and a button. The "Exam Name" field is set to "Artificial Intelligence Exam". The "Exam Date" field is set to "22/07/2025" with a calendar icon. The "Number of exams" is set to 3, indicated by a slider and the number 3. The "Difficulty" dropdown menu is set to "Medium (5-7 expansions)". The "Consistency" dropdown menu is set to "Yes". A green button labeled "Generate Exams" is at the bottom left of the form.

Figura 4.1: Interfaccia del Task 2 con le scelte definite

Una volta cliccato sul bottone **Generate Exams**, verranno generati 3 problemi nella cartella **exams**.

```

- exams
  - result
    - result_1
      - file.docx
      - file.yaml
    - result_2
      - file.docx
      - file.yaml
    - result_3
      - file.docx
      - file.yaml

```

Figura 4.2: Struttura della cartella exams dopo la generazione

Scegliendo uno dei 3 risultati prodotti, vengono riportate le informazioni contenute nel file DOCX:

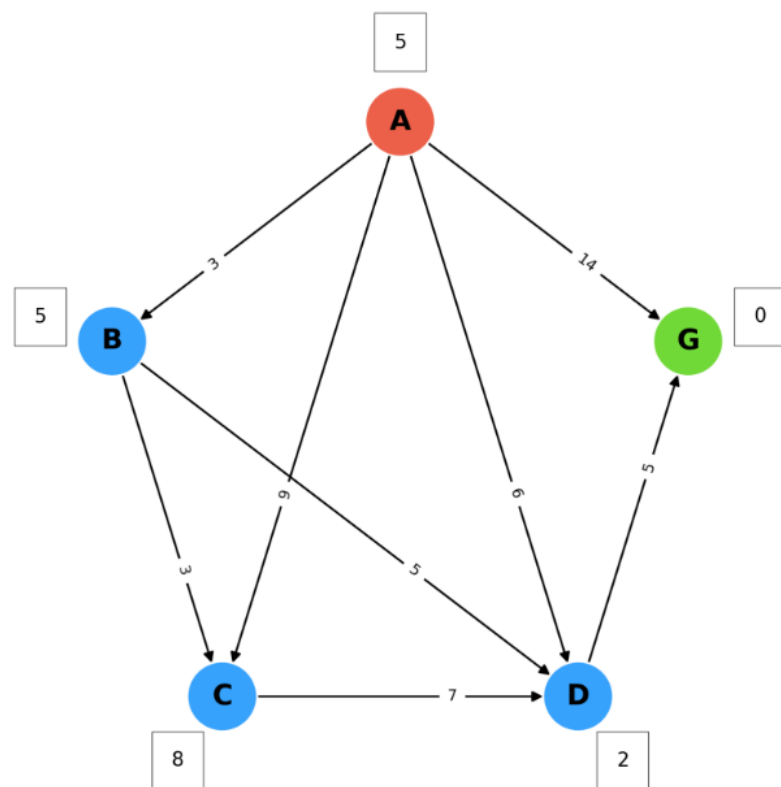


Figura 4.3: Grafo orientato generato

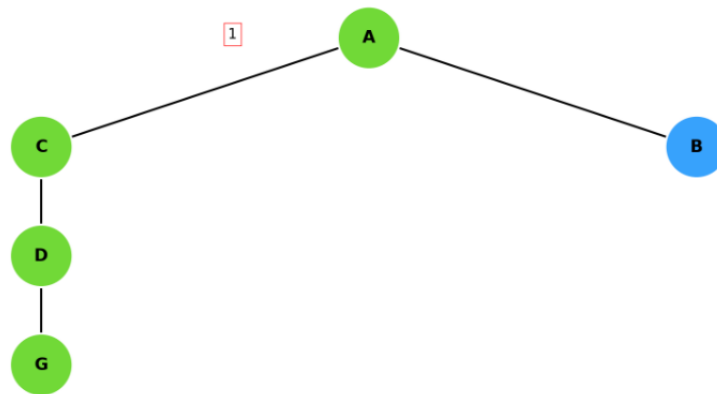


Figura 4.4: Risultato della ricerca Depth-First

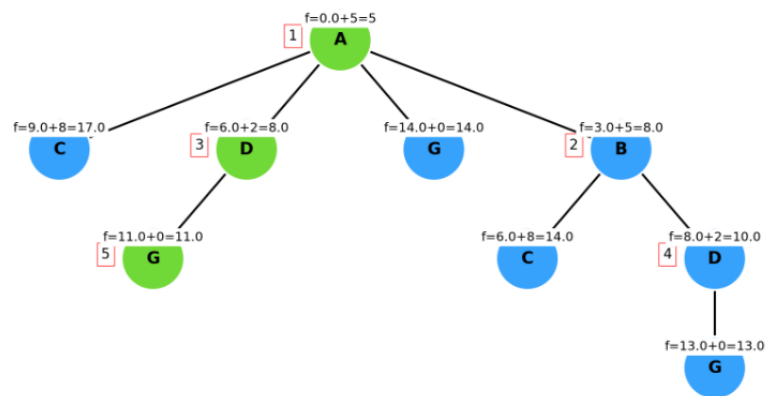


Figura 4.5: Risultato della ricerca A*

4.2 Esempi di immagini analizzate

Le immagini sottoposte ad analisi possono presentare diversi livelli di difficoltà. Una maggiore complessità è determinata da fattori quali:

- il numero di nodi presenti;
- il numero di archi e la loro densità;
- l'intersezione tra archi;
- la presenza di scritte parzialmente sovrapposte o deformate.

Di seguito viene illustrato un esempio di immagine classificabile come di difficoltà media.

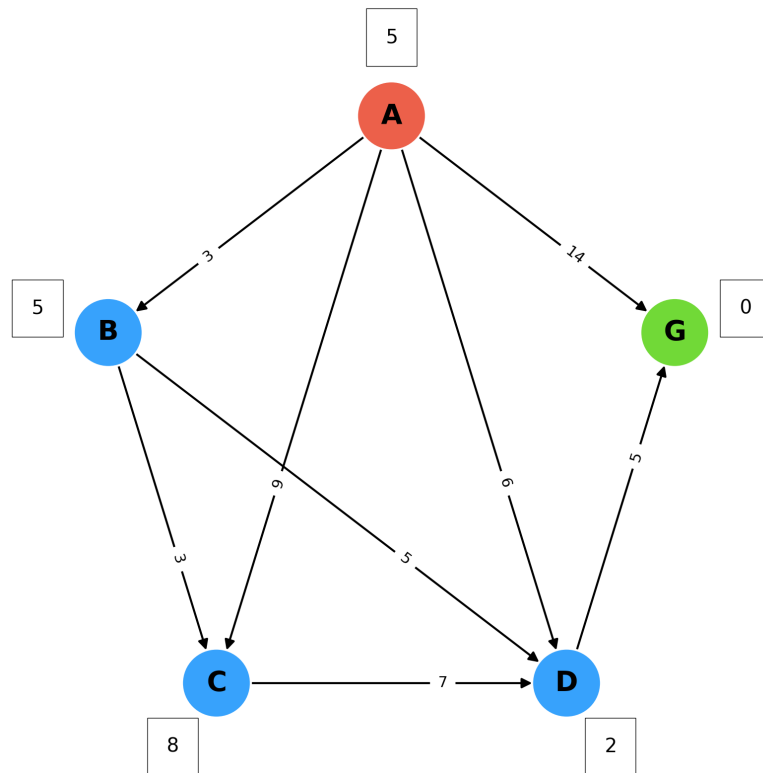


Figura 4.6: Immagine in formato .png fornita come input

Il primo output prodotto dal programma evidenzia l'individuazione dei nodi e l'estrazione dei relativi nomi tramite OCR, come mostrato in Figura 4.7.

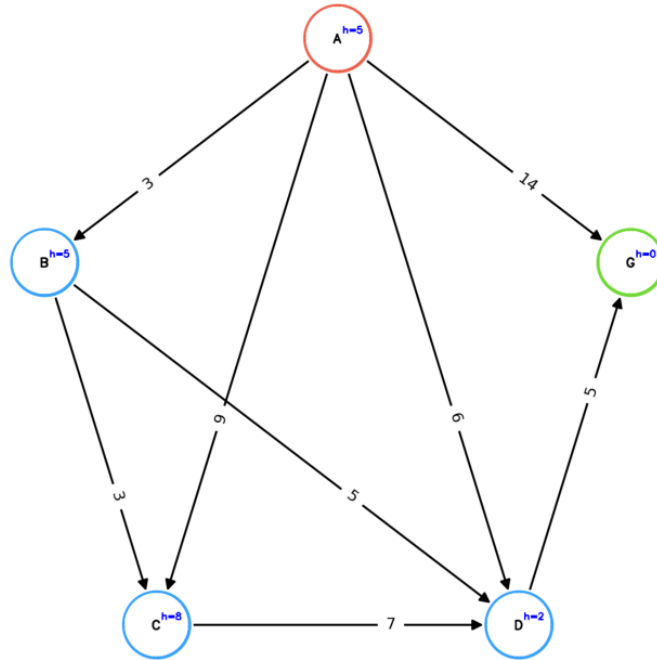


Figura 4.7: Nodi individuati (cerchi) e relativi nomi riconosciuti

Il secondo step dell'analisi consiste nell'individuazione delle linee presenti nell'immagine, che successivamente verranno raggruppate per la ricostruzione delle frecce. Tale passaggio è illustrato in Figura 4.8.

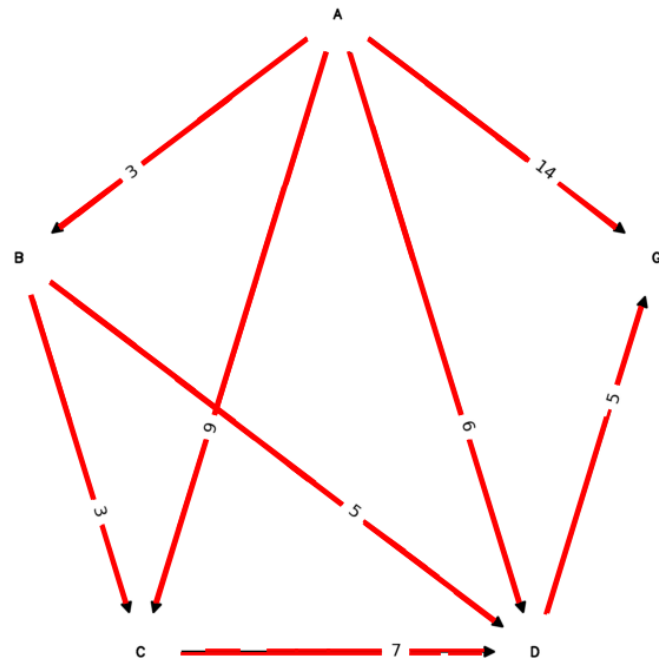


Figura 4.8: Linee individuate nell'immagine

Successivamente, le linee vengono raggruppate per caratteristiche comuni — come inclinazione e prossimità geometrica — al fine di ricostruire le frecce. Vengono inoltre rilevati gli estremi di ciascuna freccia, ovvero testa e coda, come illustrato in Figura 4.9.

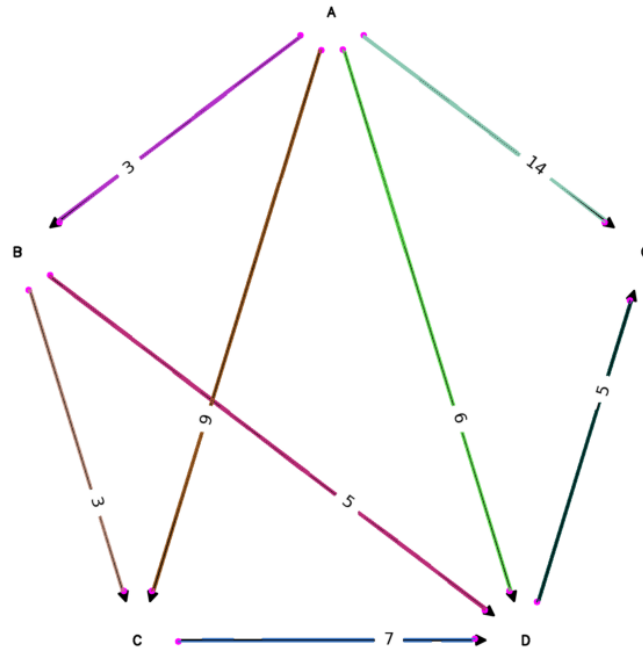


Figura 4.9: Frecce ricostruite e relativi estremi (nodi di partenza e arrivo)

Infine, viene analizzata l'area geometrica in cui è presumibilmente riportato il costo dell'arco. Questa regione viene isolata, sottoposta a OCR e il valore letto viene associato al relativo arco. Questo processo è mostrato in Figura 4.10.

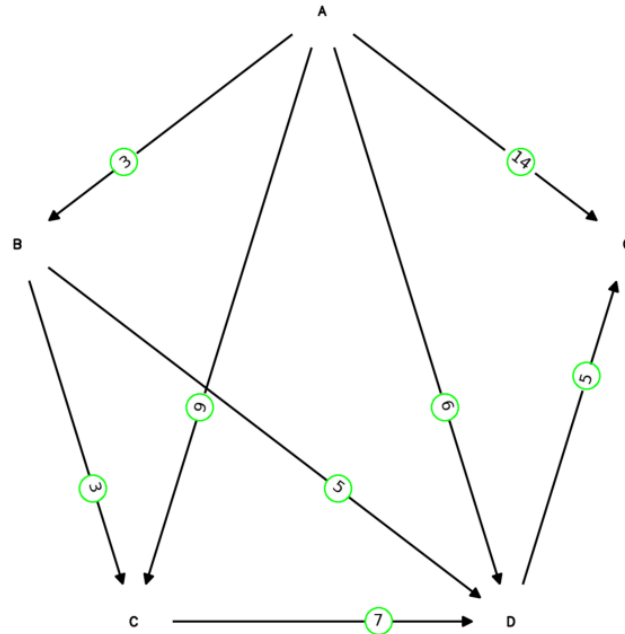


Figura 4.10: Individuazione delle aree contenenti i costi degli archi

Completata l'analisi, i dati estratti (nodi, archi, costi, euristiche) vengono utilizzati per generare un file `.yaml`, che viene poi fornito in input allo script `main.py`. Questo, come nei task precedenti, elabora il file e produce in output la soluzione in formato `.docx`, applicando sia l'algoritmo Depth-First Search sia A^* .

4.3 Analisi delle prestazioni

Task 2: Analisi della percentuale di problemi scartati. La libreria `networkx` consente di generare grafi orientati casuali specificando il numero di nodi desiderato. Tuttavia, a causa dell'elevato numero di vincoli strutturali e delle specifiche impostate dall'utente, gran parte dei grafi generati non risulta valida e viene pertanto scartata.

È stata quindi condotta un'analisi il cui obiettivo è misurare il numero medio di grafi che bisogna generare in totale per ottenerne uno valido, che quindi rispetta i requisiti prefissati. I risultati di tale analisi sono riportati nella seguente tabella. Ogni valore indica il numero medio di grafi orientati generati randomicamente per ottenere un grafo orientato che rispetti le specifiche.

Consistenza	Difficoltà		
	Facile	Media	Difficile
Sì	65–75	105–115	165–175
No	75–85	90–100	65–75

Tabella 4.1: Numero medio di grafi generati per ottenere un grafo valido

L'analisi rivela che la quantità di grafi generati randomicamente prima di ottenerne uno valido è alta, ma costante e controllabile. Molti di questi grafi vengono scartati immediatamente dopo essere stati generati, in quanto non rispettano i criteri strutturali del problema. Solo una piccola percentuale di essi viene scartata per la non concordanza sui criteri di difficoltà e consistenza.

Inoltre, c'è da considerare che, nel normale utilizzo del programma, la quantità di problemi generati è bassa, in quanto le soluzioni ottenute non necessitano di ulteriore filtraggio umano. I requisiti applicati nel *generate-and-test* sono piuttosto stringenti, per cui è sufficiente generare un numero ridotto di problemi per individuare quello ideale per il proprio utilizzo.

Infine, si rileva dall'analisi che più aumenta la difficoltà del problema, più aumenta la probabilità che venga generato un problema non consistente.

Task 3: Analisi delle imprecisioni introdotte da OpenCv. Il programma per l'analisi delle immagini è in grado di elaborare correttamente una fotografia in formato PNG, purché questa rispetti i requisiti definiti in 3.4 e la libreria OpenCV fornisca risultati compatibili con le aspettative, entro i limiti di errore tollerati e gestiti. Nel caso contrario, l'analisi non può garantire una corretta risoluzione dell'esercizio.

Nel seguito viene illustrato un esempio in cui la libreria OpenCV non riesce a rilevare correttamente tutte le linee presenti nell'immagine. In particolare, una freccia viene rilevata come più corta del reale. L'esatta individuazione degli estremi della freccia rappresenta un aspetto cruciale per determinare correttamente direzione e orientamento.

In caso di imprecisione, come nell'esempio mostrato, è probabile che venga rilevato un verso errato, compromettendo:

1. la lettura del costo associato all'arco: può essere erroneamente richiesta la lettura di un valore dove non esiste alcun costo, oppure può essere trascurato un costo realmente presente;
2. lo svolgimento corretto dell'esercizio: anche in assenza di errori di esecuzione, i dati ottenuti risulterebbero incoerenti rispetto a quelli effettivamente rappresentati nell'immagine.

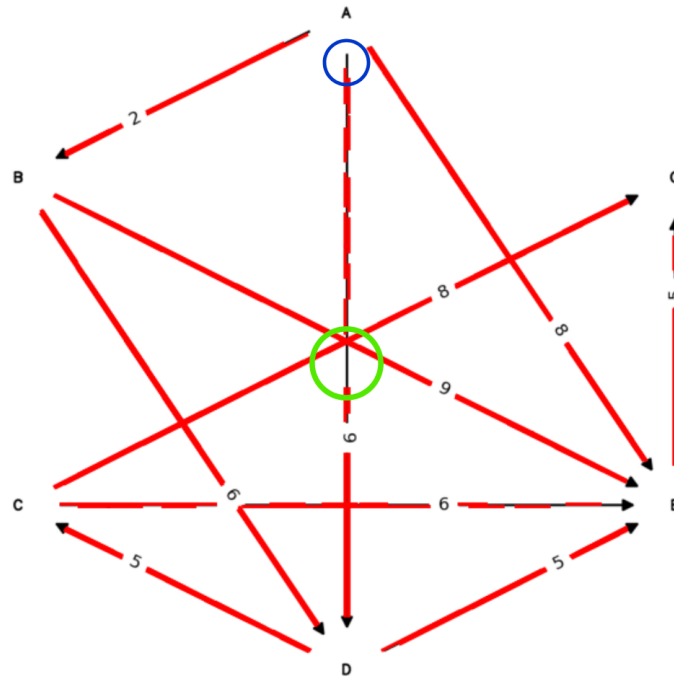


Figura 4.11: Esempio di freccia con un tratto mancante, che porta a un'interpretazione errata del verso

L'immagine evidenzia due interruzioni visibili nella freccia che collega i nodi A e D: una nella parte superiore (cerchiata in blu) e una nella parte centrale (cerchiata in verde). L'interruzione segnalata dal cerchio blu non compromette l'analisi, in quanto il programma riesce comunque a riconoscere correttamente le due sezioni spezzate come appartenenti allo stesso arco. Diversamente, l'interruzione evidenziata dal cerchio verde porta l'algoritmo a identificare erroneamente una punta anche in direzione opposta, verso il nodo A, compromettendo la corretta interpretazione della direzione della freccia.

Se invece il tratto mancante si fosse verificato in corrispondenza della reale punta della freccia, l'algoritmo non avrebbe commesso l'errore di invertire il verso. È stato scelto di mantenere questo approccio per due motivi principali:

1. l'impossibilità, in certi casi, di individuare l'intera freccia e il suo verso tramite OpenCV;
2. la tendenza osservata della libreria a rilevare con maggiore affidabilità il segmento iniziale della freccia (in prossimità della coda), ma a perdere informazioni nei tratti vicini alla punta.

Capitolo 5

Conclusioni e sviluppi futuri

Per quanto riguarda gli sviluppi futuri, il **Task 3** del progetto rimane un punto cruciale da approfondire, in quanto l'analisi dell'immagine è attualmente vincolata a un unico formato (`.png`) e a una disposizione rigida degli elementi nello spazio. In particolare, le euristiche devono essere contenute in rettangoli adiacenti ai nodi, mentre il costo degli archi deve trovarsi a circa due terzi della lunghezza della freccia, partendo dalla coda.

I possibili miglioramenti includono:

- **Supporto per formati immagine differenti** e una gestione più robusta dei disturbi visivi, come compressione, sfocatura o rumore;
- **Eliminazione dei vincoli spaziali**, permettendo l'individuazione dell'euristica anche in prossimità del nodo, senza l'obbligo di un rettangolo esplicito;
- **Riconoscimento del costo degli archi** tramite l'analisi geometrica della freccia e della sua inclinazione, accoppiando testo e arco in modo più flessibile;
- **Integrazione di approcci multipli**, ad esempio combinando OCR classico con reti neurali o modelli di visione artificiale, per aumentare l'affidabilità e la generalizzazione del riconoscimento.

Nel complesso, il progetto ha dimostrato come sia possibile generare automaticamente problemi di ricerca su spazi degli stati, partendo da input testuali o visuali, fino a ottenere la loro risoluzione automatica tramite l'algoritmo A^* . La realizzazione dell'interfaccia grafica ha facilitato l'interazione con l'utente, mentre la generazione automatica dei file YAML e l'invocazione del risolutore esterno hanno mostrato l'efficacia del sistema nel creare ed elaborare istanze complesse in modo guidato e ripetibile.

Il lavoro svolto costituisce quindi una base solida per future estensioni e miglioramenti, sia dal punto di vista della robustezza degli strumenti di analisi che dell'esperienza utente.

Bibliografia

- [1] EITCA Academy. *What is JupyterLab and how can it be accessed in a Deep Learning VM*. <https://it.eitca.org/artificial-intelligence/eitc-ai-gcml-google-cloud-machine-learning/advancing-in-machine-learning/deep-learning-vm-images/examination-review-deep-learning-vm-images/what-is-jupyterlab-and-how-can-it-be-accessed-in-a-deep-learning-vm/>. Consultato il 5 luglio 2025. 2023.
- [2] GeekandJob. *OpenCV*. Accessed: 2025-06-04. n.d. URL: <https://www.geekandjob.com/wiki/opencv>.
- [3] IBM. *Cos'è il riconoscimento ottico dei caratteri (OCR)?* Accessed: 2025-07-01. 2024. URL: <https://www.ibm.com/it-it/think/topics/optical-character-recognition>.
- [4] Peter Russell Stuart e Norvig. *Intelligenza Artificiale: un approccio moderno*. 4^a ed. Pearson Education, 2021.